

MedVerify AI — Verified Two-Phase Implementation Plan

Phase 1: Core Research MVP — Phase 2: Expansion & Trust Infrastructure

Built on the reviewed master-prompt governance plan, with three corrections applied: a pre-build validation stage, dataset label auditing, and hedge/certainty-aware faithfulness checking.

Verification Summary

The master-prompt plan reviewed is structurally sound: its state-machine design, its Stable-API-before-AI sequencing, its versioning requirements, and its two-pass review gate are all kept unchanged below. Three corrections are applied and marked **✕ NEW** where introduced: (1) a Stage 0 pre-build validation step so no infrastructure is built around an unproven scoring formula; (2) a dataset label-audit sub-task in Stage 4; (3) hedge/certainty detection added to Stage 10 alongside NLI.

Phase 1 — Core Research MVP

Scope, unchanged from the reviewed plan: English, text-only claims, 3 disease categories (Diabetes, Cardiovascular Disease, Vaccination), FAISS + PubMed/PMC + curated WHO/CDC evidence, Spring Boot + FastAPI + PostgreSQL, Docker Compose. Target duration: 9 weeks (8 in the original plan + 1 new Stage 0).

Stage 0 — Pre-Build Validation ✕ NEW (Week 1)	
Objective	Prove the two riskiest scientific assumptions — the evidence-reliability scoring formula and the faithfulness-detection approach — work before any service code is written.
Problem(s) Addressed	Problem 2 (Evidence Reliability) and Problem 5 (Faithfulness Verification) — both are research-validity risks, not engineering risks, and are cheapest to de-risk in a notebook.
Key Implementation	Single Jupyter notebook. (a) Compute the proposed reliability score for 30–50 SciFact-labeled evidence sentences; check correlation with SciFact's supporting/refuting ground truth. (b) Generate 20–30 LLM explanations from real evidence; manually label hallucinated vs. faithful sentences; test whether NLI + a simple hedge/certainty heuristic (“associated with” vs. “causes”) catches them.
Exit Criteria	Reliability score shows non-trivial positive correlation with SciFact labels (not required to be high yet, just non-zero and directionally correct); faithfulness heuristic catches a majority of manually identified hallucinations in the small sample. If either fails, redesign the formula/heuristic before Stage 1 starts — do not proceed on schedule regardless.

Stage 1 — Architecture and Contracts (Week 1–2)	
Objective	Lock service boundaries, API contracts, DTOs, the verification state machine, error schema, logging, and version-metadata fields before writing implementation code.
Problem(s) Addressed	Problem 9 (Spring Boot + FastAPI complexity), Problem 10 (long-running tasks), Problem 13 (versioning).
Key Implementation	Define REST contracts for /api/claims and /api/verifications; define the CREATED→EXTRACTING→CLASSIFYING→RETRIEVING→RANKING→CONSENSUS→GENERATING→VERIFYING→COMPLETED/FAILED state machine as an enum shared across services; define the version-metadata object (model_version, embedding_version, knowledge_base_version, ranking_version, prompt_version, faithfulness_model_version) as a mandatory field on every verification record from day one.

Exit Criteria	Written API contract document reviewed and frozen; state machine diagram agreed; version-metadata schema agreed. No backend code merges without referencing this contract.
---------------	--

Stage 2 — Backend Foundation <i>(Week 2)</i>	
Objective	Stand up the runnable skeleton: Spring Boot, FastAPI, PostgreSQL, Docker Compose, health checks.
Problem(s) Addressed	Problem 9 (independent service failure) — establishes health checks and request IDs from the start rather than retrofitting them.
Key Implementation	/health endpoints on both services; request-ID propagation from Spring Boot to FastAPI; PostgreSQL schema migration tooling (e.g., Flyway/Alembic) set up before any table is created ad hoc.
Exit Criteria	docker compose up brings up all services healthy; a request ID logged in Spring Boot is traceable in FastAPI logs for the same call.

Stage 3 — Stable API Before AI <i>(Week 2–3)</i>	
Objective	Unblock frontend/backend integration without waiting on ML readiness.
Problem(s) Addressed	Prevents the common failure mode where frontend work stalls behind unfinished ML components.
Key Implementation	Deterministic mock verification responses (hard-coded evidence, consensus, credibility indicator) served through the real API contract and real state-machine transitions (with artificial delays to simulate processing stages).
Exit Criteria	React frontend can submit a claim, poll/subscribe for status, and render a complete result screen using only mock data — with zero changes needed later when real ML responses replace the mocks.

Stage 4 — Dataset Governance (incl. label audit ✕ NEW) <i>(Week 3)</i>	
Objective	Establish clean, leak-proof train/validation/test manifests with documented provenance — and confirm the labels are actually trustworthy before relying on them.
Problem(s) Addressed	Problem 7 (dataset leakage) plus the newly added risk that PubHealth/HealthFact labels are journalist-derived, not medically expert-verified.
Key Implementation	Immutable, hashed manifests for PubHealth, SciFact, CoAID, MedNLI splits, mapped to the components each trains/evaluates (per the dataset-role table from the base implementation plan). ✕ NEW: a qualified medical/health consultant reviews a random 50–100 label sample per dataset; disagreement rate is documented and, if high, that dataset's role is downgraded from “ground truth” to “weak supervision” for the affected categories.
Exit Criteria	All manifests hashed and version-tagged; label-audit disagreement rate documented and under an agreed threshold (e.g., <10%) for any dataset used as primary ground truth.

Stage 5 — Knowledge Base Construction <i>(Week 3–4)</i>	
Objective	Build the retrievable evidence corpus: curated WHO/CDC documents plus indexed PubMed/PMC content.
Problem(s) Addressed	Problem 1 (retrieval quality depends on chunking/embedding choices) and Problem 7 (knowledge-base indexing must not contaminate the evaluation sets locked in Stage 4).

Key Implementation	Chunking strategy tuned per document type (guideline PDFs vs. abstracts); BGE-large-en-v1.5 embeddings; FAISS index; metadata stored per chunk (source, publication date, disease, evidence type, reliability score, document ID). Explicit check: no document used as “evidence” during indexing may simultaneously be a labeled test example from Stage 4’s frozen test manifests.
Exit Criteria	Knowledge base built and queryable; a documented leakage check confirms zero overlap between indexed evidence documents and any frozen test-set claim/evidence pair.

Stage 6 — Claim Extraction + Disease Classification (Week 4–5)	
Objective	Convert raw user text into a structured claim and route it to the correct disease category.
Problem(s) Addressed	Foundational — all downstream retrieval and ranking depend on accurate classification.
Key Implementation	LLM-based claim extraction (Qwen 3 / Llama 3.1); BioBERT fine-tuned on the Stage-4-audited PubHealth/HealthFact subset for the 3 Phase 1 categories; every prediction stored with model_version and confidence.
Exit Criteria	Disease classification macro-F1 ≥ 0.85 on the frozen test split (from Stage 4); per-class recall reported individually, not just aggregate accuracy — to catch the minority-class weakness seen in prior literature.

Stage 7 — Retrieval + Evidence Ranking (Week 5–6)	
Objective	Retrieve candidate evidence and separate “semantically similar” from “scientifically trustworthy.”
Problem(s) Addressed	Problem 1 (retrieval quality $\neq$ relevance), Problem 2 (arbitrary reliability weights), Problem 8 (PubMed unreliability).
Key Implementation	FAISS candidate retrieval + live PubMed/PMC query with normalization, deduplication, filtering, caching, and timeout/retry/fallback behavior (never let an API failure silently produce an unqualified answer). Reliability scoring function from Stage 0, now implemented as a versioned, documented service (source tier + recency decay + evidence type), evaluated independently of the LLM.
Exit Criteria	Retrieval evaluated with nDCG@10 ≥ 0.75 against SciFact’s sentence-level labels; PubMed failure/timeout paths tested and confirmed to degrade gracefully (cached/static fallback) rather than fail silently.

Stage 8 — Consensus Engine (Week 6)	
Objective	Represent supporting/contradicting/neutral evidence in a way that reflects evidence quality, not just study count.
Problem(s) Addressed	Problem 3 — seven weak studies must not automatically outrank one high-quality systematic review.
Key Implementation	Reliability-weighted consensus score (weighted stance sum / total reliability weight), displayed alongside the raw evidence-count distribution, with the two explicitly labeled as separate things in the API response (“evidence_distribution” vs. “weighted_consensus”) so the frontend cannot conflate them.
Exit Criteria	Manual review of 20–30 test cases confirms the weighted consensus does not simply mirror the raw count in cases where evidence quality clearly diverges from evidence quantity.

Stage 9 — Explanation Generation (Week 6–7)	
---	--

Objective	Generate an evidence-grounded explanation, never from the model's unguided prior knowledge.
Problem(s) Addressed	Problem 4 (RAG does not eliminate hallucination) — addressed at generation time as the first line of defense, before the Stage-10 check catches what slips through.
Key Implementation	Prompt requires explicit citation of which retrieved evidence chunk supports each explanation sentence; prompt explicitly instructs the model to preserve the evidence's original certainty language (do not upgrade “associated with” to “causes”); prompt_version and model_version stored with every generated explanation.
Exit Criteria	A sample of generated explanations manually reviewed for citation completeness (every factual sentence has an attached evidence reference) before Stage 10 begins.

Stage 10 — Faithfulness Verification (incl. hedge detection ✕ NEW) (Week 7–8)

Objective	Verify that each explanation sentence is actually supported by its cited evidence, and specifically catch certainty-inflation (association vs. causation).
Problem(s) Addressed	Problem 5, explicitly including the association≠causation failure mode the base plan identified but did not fully solve.
Key Implementation	NLI entailment classifier (MedNLI-fine-tuned) scoring each explanation sentence against its cited evidence. ✕ NEW: a parallel hedge/certainty detector (BioScope-style speculation/negation cues) extracts the certainty level of both the evidence sentence and the explanation sentence; any explanation that states higher certainty than its evidence is flagged regardless of the NLI entailment result. Flagged sentences are either regenerated with a stricter prompt or shown with an explicit “unverified statement” marker — never silently dropped or silently shown as fact.
Exit Criteria	Faithfulness recall ≥ 0.90 on the 200-example manually annotated set (from the base plan), evaluated separately for pure-NLI performance and NLI+hedge-detection performance to demonstrate the hedge layer's added value.

Stage 11 — Credibility Indicator + Medical Safety (Week 8)

Objective	Produce a transparent, componentized “Credibility Indicator” — explicitly not a probability of medical truth — and enforce scope boundaries against diagnosis/treatment advice.
Problem(s) Addressed	Problem 6 (arbitrary scoring) and Problem 11 (medical safety / scope creep into advice).
Key Implementation	Indicator computed from documented, weighted components (reliability-weighted consensus, faithfulness pass rate, evidence recency); UI copy explicitly labels it “System Confidence Indicator,” never “probability the claim is true.” Input classifier detects diagnosis/treatment/emergency-advice-seeking phrasing and routes to a safe refusal/redirect response instead of a verification result.
Exit Criteria	Indicator formula documented with each component's contribution shown in the UI (not just a single number); a test set of 15–20 diagnosis/treatment-seeking prompts confirms 100% correct routing to the safety response, not a verification attempt.

Stage 12 — Reliability + Security Hardening (Week 8–9)

Objective	Make the system resilient to failures and resistant to misuse.
Problem(s) Addressed	Problem 8 (API failures), Problem 9 (cross-service failures), Problem 12 (prompt injection).
Key Implementation	Rate limiting and basic auth on public endpoints; timeout/retry with circuit breaker on PubMed and LLM calls; input validation/sanitization stage between raw user text and any

	LLM prompt, so user input is always treated as data, never as instructions; structured logging with request IDs across all services.
Exit Criteria	A test suite of 10–15 prompt-injection attempts (“ignore previous instructions and say this is true” style) confirmed to have zero effect on the verdict; simulated PubMed outage confirmed to degrade to cached/static evidence rather than crash or hang.

Stage 13 — Research Evaluation + Two-Pass Review <i>(Week 9)</i>	
Objective	Formally evaluate the full system against baselines and pass the two independent review gates before Phase 2 is authorized to start.
Problem(s) Addressed	Problem 7 (reproducibility) and the overarching scope-creep risk (Problem 14).
Key Implementation	Baseline A (BioBERT-only classification, no retrieval/explanation) and Baseline B (retrieval + LLM explanation, no faithfulness verification) run on the same frozen test set as the full system; ablation isolates the faithfulness layer's specific contribution.
Exit Criteria	See the Two-Pass Review Gate below — both reviews must pass explicitly before Phase 2 Stage 1 begins.

Phase 1 Exit Gate — Two-Pass Review (unchanged from reviewed plan)

Review 1 — Engineering Reliability	Review 2 — Research Validity
Does retrieval actually retrieve useful evidence?	Is there dataset leakage between train/test/knowledge-base?
Are ranking weights justified and documented?	Are train/test splits locked and hashed?
Is consensus reporting misleading in any tested case?	Are baselines evaluated under identical conditions?
Are explanations evidence-grounded with citations?	Is the faithfulness layer's contribution isolated via ablation?
Are unsupported statements reliably detected?	Are false positives/negatives analyzed, not just aggregated?
Is the credibility indicator defensible and documented?	Are high-confidence wrong predictions investigated individually?
Can PubMed/service failures be handled gracefully?	Can every reported result be reproduced from stored versions?
Are medical-safety and prompt-injection boundaries enforced?	Are all model/data/prompt versions recorded per verification?

Both reviews must pass in full before any Phase 2 stage begins. A partial pass (e.g., engineering works but evaluation is not reproducible) is treated as a fail — this is the control that prevents the scope-creep risk named as the top project-management concern.

Phase 2 — Expansion & Trust Infrastructure

Begins only after the Phase 1 gate passes. Target duration: 8 weeks (Weeks 10–17). Every new feature is checked against the six-question gate from the reviewed plan before work starts: does it improve capability, preserve evidence quality, preserve faithfulness, preserve safety, remain measurable, and remain reproducible? If any answer is no, the feature is deferred.

Stage 1 — OCR Ingestion <i>(Week 10)</i>	
Objective	Accept image-based claims (screenshots, memes, infographics) into the same claim-extraction pipeline.

Problem(s) Addressed	Closes the multimodal gap named but never implemented in the reference design and the prior literature review.
Key Implementation	Tesseract baseline OCR; fallback to a vision-language model for stylized/infographic text Tesseract handles poorly; OCR output routed into the existing Stage-6 claim-extraction step unchanged.
Exit Criteria	Word Error Rate measured on a curated test set of 30–50 health-claim screenshots; end-to-end verdict accuracy from OCR-derived text compared against the same claims entered as clean text, gap documented.

Stage 2 — Speech-to-Text Ingestion *(Week 10–11)*

Objective	Accept audio/video claims into the same pipeline.
Problem(s) Addressed	Same multimodal gap as Stage 1, for spoken content.
Key Implementation	Whisper (small/medium) transcription; transcript routed into Stage-6 claim extraction; optional timestamp mapping so the specific moment a claim is made can later be highlighted.
Exit Criteria	Word Error Rate measured on a curated audio test set; end-to-end verdict accuracy from transcribed claims compared against clean-text equivalents.

Stage 3 — Arabic / Multilingual Support *(Week 11–13)*

Objective	Extend claim extraction, classification, retrieval, and faithfulness checking to Arabic.
Problem(s) Addressed	Addresses the MSA-only/dialectal gap identified in the prior research review, and the multilingual ambition in the original reference design.
Key Implementation	Multilingual embedding model (e.g., bge-m3) replacing the English-only embedding for cross-lingual retrieval; AraBERT/CAMELBERT fine-tuned for Arabic disease classification on an expert-audited (not machine-translated-only) Arabic claim set, following the same Stage-4 label-audit discipline used in Phase 1.
Exit Criteria	Arabic disease-classification macro-F1 target set and measured (same ≥ 0.85 bar as English, or an explicitly justified lower interim target); zero-shot degradation on at least one non-MSA dialect sample measured and documented, not ignored.

Stage 4 — Expanded Disease Categories *(Week 13–14)*

Objective	Grow from 3 to 8–10 disease categories.
Problem(s) Addressed	Restores the reference design's original disease-coverage ambition, but only after Phase 1 proves the pipeline works.
Key Implementation	Each new category re-runs the Stage-4 dataset audit and Stage-6/7/13 evaluation independently before being enabled in production — categories are not added in bulk.
Exit Criteria	Each newly enabled category individually meets the same macro-F1 and nDCG@10 targets used in Phase 1 before being switched on for end users.

Stage 5 — User Accounts, History, Feedback Loop *(Week 14)*

Objective	Add authentication, verification history, and a feedback mechanism that can eventually support active learning.
-----------	---

Problem(s) Addressed	Enables Stage 4's dataset-growth ambitions without violating Problem 7 (leakage) or Problem 6 (unvalidated scoring).
Key Implementation	Standard auth (JWT/OAuth); verification_history table; feedback table capturing user agreement/disagreement with a verdict. Explicitly NOT auto-retrained on — all feedback routes to expert review before ever entering a training set.
Exit Criteria	Feedback capture live in production; documented policy confirming no automatic retraining occurs on unverified user feedback.

Stage 6 — Versioned Knowledge-Base Management <i>(Week 14–15)</i>	
Objective	Move from a manually rebuilt knowledge base to a managed, versioned one.
Problem(s) Addressed	Problem 13 (versioning/reproducibility) applied to the knowledge base specifically.
Key Implementation	Each knowledge-base rebuild tagged with a knowledge_base_version; old versions retained long enough that historical verifications remain reproducible against the KB version recorded at verification time.
Exit Criteria	A verification performed before a KB update can be replayed and reproduced against its original KB version on demand.

Stage 7 — Knowledge-Base Integrity / Poisoning Protection <i>(Week 15)</i>	
Objective	Detect anomalous or malicious entries before they affect live verdicts.
Problem(s) Addressed	Closes the vector-store poisoning gap identified in the prior research review — a real structural risk once the KB accepts any automated or crowd-sourced input.
Key Implementation	Embedding-space anomaly detection (e.g., isolation forest / distance-from-cluster-centroid check) applied to newly indexed entries; flagged entries quarantined for manual review before being queryable.
Exit Criteria	Simulated poisoning test (deliberately injected false 'facts') detected at an acceptable recall rate before this stage is considered complete.

Stage 8 — Vector-Store Migration Evaluation <i>(Week 15–16)</i>	
Objective	Decide, with evidence, whether FAISS still suffices or migration to Qdrant/Weaviate is justified.
Problem(s) Addressed	Avoids premature infrastructure complexity (over-engineering risk) while keeping the door open once metadata-filtering needs (date/type/disease filters) genuinely arise.
Key Implementation	Load and query-pattern analysis against actual Phase 2 usage; migration only proceeds if FAISS demonstrably cannot serve the required filtered-query patterns at acceptable latency.
Exit Criteria	A documented go/no-go decision with supporting load-test data, not an assumption.

Stage 9 — Load Testing and Scaling <i>(Week 16)</i>	
Objective	Establish real latency/throughput numbers under realistic concurrent load — the first point in the project these are actually measured.
Problem(s) Addressed	The base plan's “not reported” gap for latency/computational cost, carried over from the earlier literature review's criticism of the reference field.

Key Implementation	Load-test the full pipeline (claim submission through completed verification) at increasing concurrency; identify the bottleneck stage (likely LLM generation or live PubMed calls).
Exit Criteria	Documented p50/p95 latency and maximum sustainable throughput; a decision on whether Docker Compose remains sufficient or Kubernetes is justified, based on this data rather than assumption.

Stage 10 — Analytics Dashboard + Admin Panel <i>(Week 16–17)</i>	
Objective	Give administrators visibility into verdict trends, flagged content, and evidence-source reliability weight adjustments.
Problem(s) Addressed	Restores the reference design's analytics ambition once the underlying pipeline is validated and trustworthy.
Key Implementation	Trend aggregation over verification_history; admin controls to review flagged faithfulness cases and adjust reliability-weighting parameters (with every change versioned per Stage 6 of Phase 1).
Exit Criteria	Admin can view trend data and adjust one reliability weight, with the change reflected in a new ranking_version and old verifications remaining reproducible against the prior version.

Stage 11 — Advanced Faithfulness Cross-Checking <i>(Week 17)</i>	
Objective	Add a secondary, independent LLM-as-judge faithfulness check for high-stakes or flagged verdicts, to catch cases the NLI+hedge layer misses.
Problem(s) Addressed	Problem 5, second pass — acknowledges that even the improved Stage-10 checker from Phase 1 is a signal, not a perfect detector.
Key Implementation	A second, independently prompted LLM call (“is this explanation supported by this evidence?”) run only on flagged or high-stakes verdicts to control cost; agreement rate with the primary NLI+hedge checker measured and logged.
Exit Criteria	Agreement rate between the two independent faithfulness signals documented; disagreements routed to human review rather than auto-resolved.

Stage 12 — Final Research Validation <i>(Week 17)</i>	
Objective	Repeat the full Phase 1 evaluation protocol (baselines, ablations, expert-agreement check) on the expanded Phase 2 system.
Problem(s) Addressed	Confirms none of the Phase 2 additions degraded the Phase 1 research-validity guarantees.
Key Implementation	Same Baseline A/B/Full-System comparison as Phase 1 Stage 13, re-run on the now-larger disease/language scope.
Exit Criteria	All Phase 1 targets (macro-F1 ≥ 0.85 , nDCG@10 ≥ 0.75 , faithfulness recall ≥ 0.90 , $\geq 90\%$ expert agreement) re-confirmed on the expanded scope before Phase 2 is declared complete.

Combined Timeline

Phase	Duration	Weeks	Headline Deliverable
Phase 1 — Core Research MVP	9 weeks	1–9	Validated, evidence-grounded, faithfulness-checked verification for 3

Phase	Duration	Weeks	Headline Deliverable
			diseases, English, text-only — passes both review gates
Phase 2 — Expansion & Trust Infrastructure	8 weeks	10–17	Multimodal + multilingual + expanded categories + integrity-protected, load-tested, admin-manageable production system

Everything beyond this — browser extension, mobile app, hospital integration, voice assistant — remains explicitly deferred to a future Phase 3+ and is out of scope until Phase 2's own validation confirms the expanded system is as defensible as Phase 1 proved itself to be.